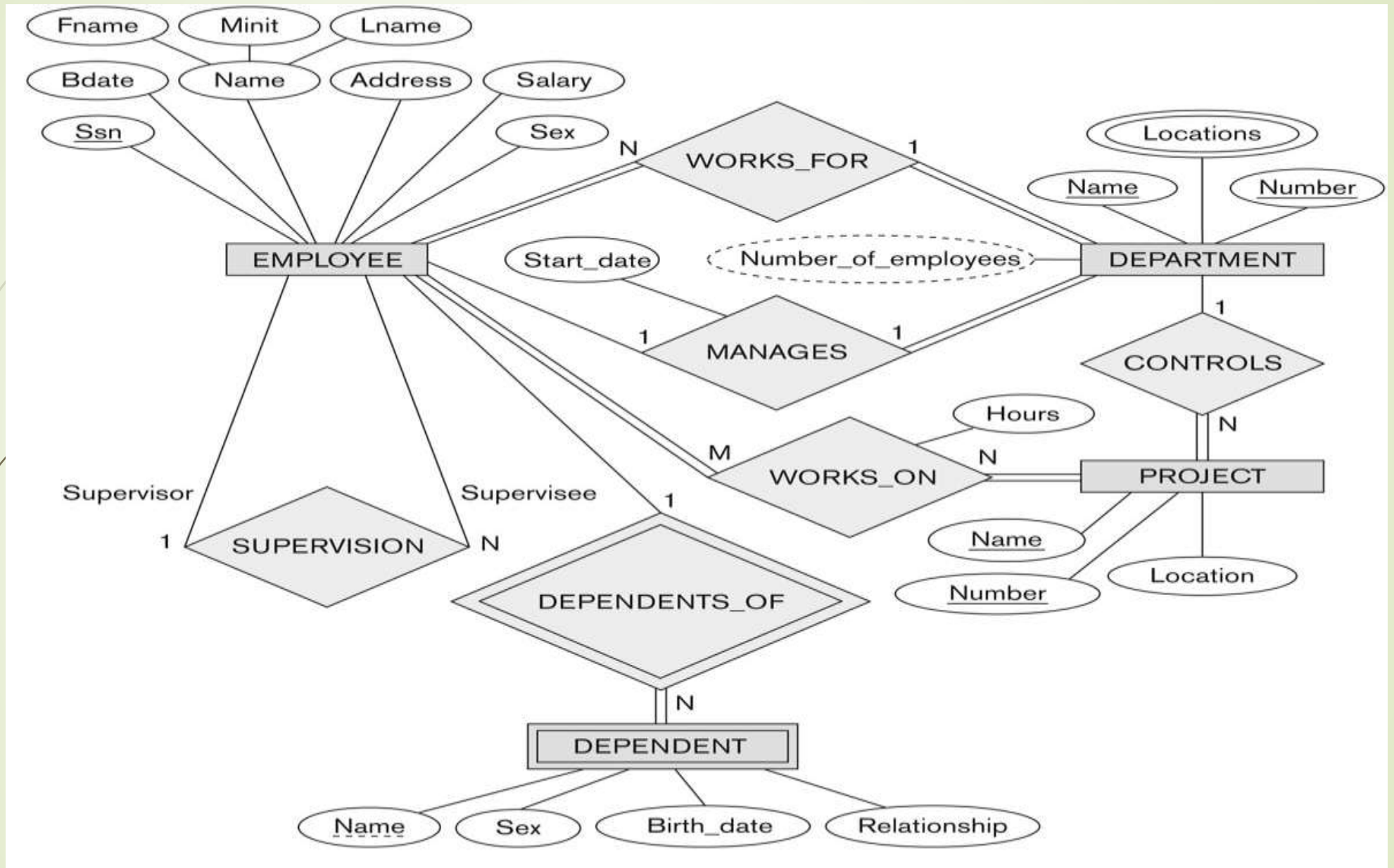


RELATIONAL DATABASE DESIGN BY ER- AND EER-TO-RELATIONAL MAPPING







ER-TO-RELATIONAL MAPPING ALGORITHM

- Seven steps to translate an ER diagram into a collection of tables with associated constraints.
 1. Mapping of Regular Entity Types
 2. Mapping of Weak Entity Types
 3. Mapping of Binary 1:1 Relationship Types
 4. Mapping of Binary 1:N Relationship Types
 5. Mapping of Binary M:N Relationship Types
 6. Mapping of Multivalued Attributes
 7. Mapping of N-ary Relationship Types

Step 1: Mapping of Regular Entity Types

- For each regular entity type E in the ER schema, create a relation R that includes all the **simple attributes** of E.
- Choose one of the key attributes of E as a **primary key** for R. For each regular entity type E in the ER schema, create a relation R that includes all the simple attributes of E.
- Choose one of the key attributes of E as a primary key for R.

Regular entities:

- EMPLOYEE
- DEPARTMENT
- PROJECT

```
EMPLOYEE(  
    Ssn PRIMARY KEY,  
    Fname,  
    Minit,  
    Lname,  
    Bdate,  
    Address,  
    Sex,  
    Salary  
)
```

Composite attribute Name is split into Fname, Minit, Lname .

- 
- A **composite attribute** is an attribute made up of smaller (simple) attributes
 - If such a composite attribute is chosen as the **primary key**
 - Then **all its simple parts together become the primary key**
 - You do **NOT** keep the composite attribute as a single column

- **Name** → composite attribute
- **Fname**, **Minit**, **Lname** → simple attributes

Assume **Name** is chosen as the **primary key**.

```
EMPLOYEE(  
    Fname,  
    Minit,  
    Lname,  
    Salary,  
    Dept,  
    PRIMARY KEY (Fname, Minit, Lname)  
)
```

Step 2: Mapping of Weak Entity Type

- For each weak entity type W with owner entity E, **create a relation R and include all simple attributes of W.**

Create a table for the weak entity and add all its attributes.

Example:

DEPENDENT attributes:

- Name
- Sex
- Birth_date
- Relationship

- 
- 
- Include as foreign key attributes of R, the primary key of the owner entity.

Add the **EMPLOYEE** primary key into the **DEPENDENT** table.

Example:

EMPLOYEE primary key = Ssn

```
DEPENDENT(  
    Essn,                ← foreign key  
    Dependent_name,  
    Sex,  
    Birth_date,  
    Relationship  
)
```

- 
- 
- The primary key of R will be the combination of the primary key of the owner and the partial key of the weak entity.

- DEPENDENT alone is **not unique**
- Combine:
 - Owner key (`Essn`)
 - Partial key (`Dependent_name`)

- Same employee cannot have two dependents with same name
- Different employees can have dependents with same name

```
DEPENDENT(  
    Essn,  
    Dependent_name,  
    Sex,  
    Birth_date,  
    Relationship,  
    PRIMARY KEY (Essn, Dependent_name),  
    FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn)  
)
```


- 
- If a weak entity E2 depends on another weak entity E1, map E1 first.

You must **know the primary key of the owner first** before mapping the next weak entity.

(Optional Company Example)

EMPLOYEE → DEPENDENT → DEPENDENT_MEDICAL

You must map:

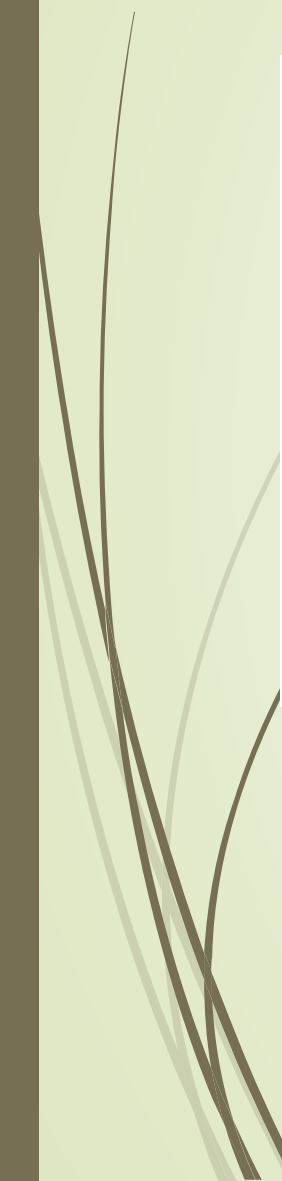
- 1. EMPLOYEE**
- 2. DEPENDENT**
- 3. DEPENDENT_MEDICAL**

Step 3: Mapping of Binary 1:1 Relationship

- One entity instance is related to **only one** instance of another entity and vice-versa

Relationship: MANAGES

- One EMPLOYEE manages one DEPARTMENT
- One DEPARTMENT is managed by one EMPLOYEE
- Attribute of relationship: Start_date

- 
- 
- There are three possible approaches:
 1. Foreign key approach
 2. Merged relation approach
 3. Cross-reference or relationship relation approach

Foreign Key Approach

- Add the **primary key of one entity** as a **foreign key** in the other entity
- Also add relationship attributes there

```
EMPLOYEE(  
    Ssn PRIMARY KEY,  
    Name,  
    Salary  
)
```

```
DEPARTMENT(  
    Dnumber PRIMARY KEY,  
    Dname,  
    Mgr_ssn,  
    Mgr_start_date,  
    FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)  
)
```

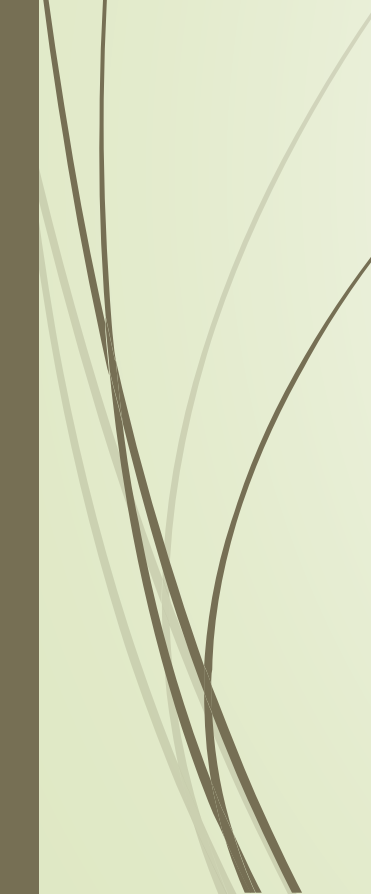


Merged Relation Approach

- Merge both entities **and the relationship** into **one table**
- Used **only when both sides have total participation**

- **Example:**

- Every employee must manage a department
Every department must have a manager



DEPARTMENT

Dname	<u>Dnumber</u>
-------	----------------

DEPARTMENT_W_MANAGER

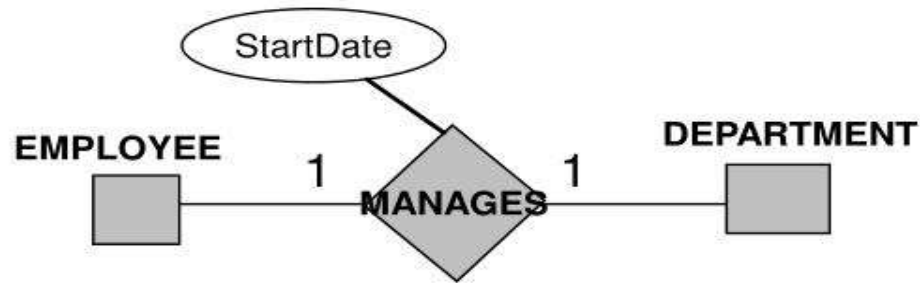
_____ (#)



Cross-Reference (Relationship Relation) Approach

- Create a separate table for the relationship
- Store keys of both entities
- Usually used for binary M:N relationships

Cross-Reference Approach



EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary
-------	-------	-------	------------	-------	---------	-----	--------

DEPARTMENT

Dname	<u>Dnumber</u>
-------	----------------

Cross-reference approach:

MANAGES

<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
----------------	---------	----------------

MANAGES(Dnumber) REFERENCES DEPARTMENT(Dnumber)

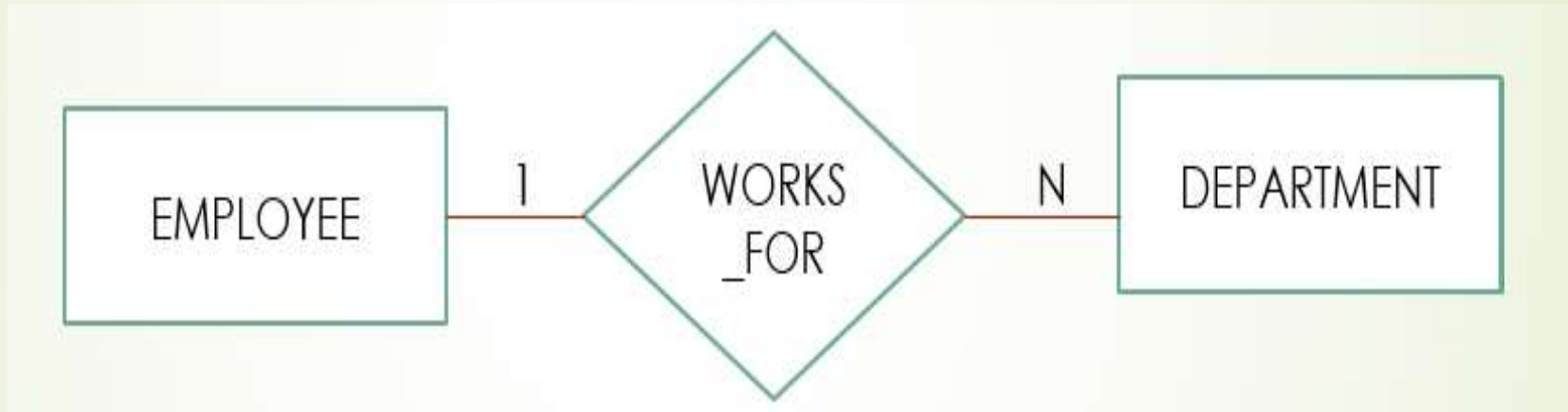
MANAGES(Mgr_ssn) REFERENCES EMPLOYEE(Ssn)



Step 4: Mapping of Binary 1:N Relationship Types

- There are two possible approaches:
 1. The foreign key approach
 2. The relationship relation approach

Example



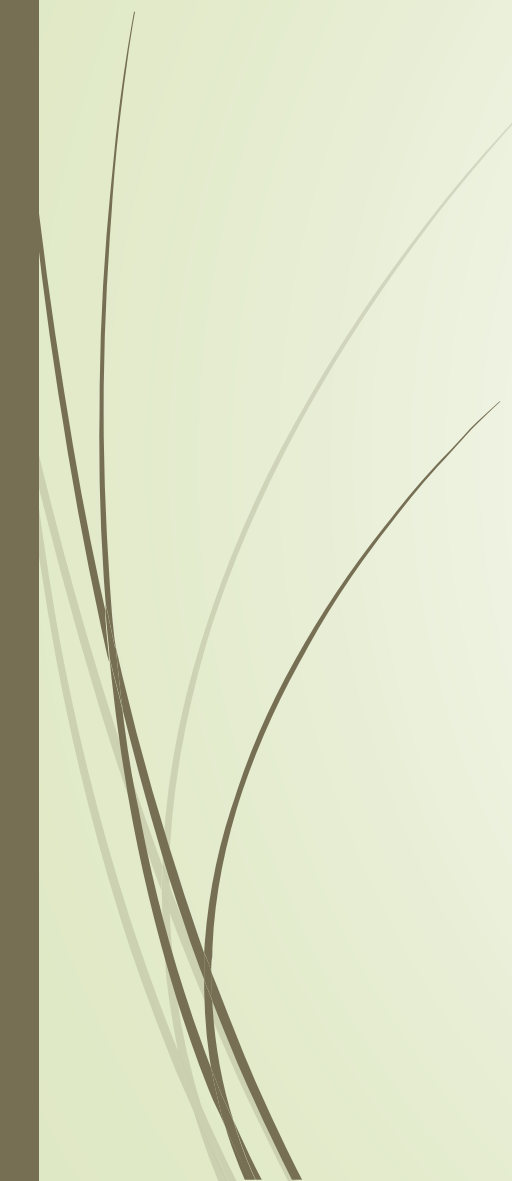
- One **DEPARTMENT** has **many EMPLOYEES**
- Each **EMPLOYEE** works for **one DEPARTMENT**



The foreign key approach

- Add the **primary key** of the **1-side** entity
- As a **foreign key** in the **N-side** entity
- Add relationship attributes (if any) to the N-side

- 1-side → DEPARTMENT
- N-side → EMPLOYEE
- Primary key of DEPARTMENT = Dnumber



DEPARTMENT (

Dnumber PRIMARY KEY,

Dname

)

EMPLOYEE (

Ssn PRIMARY KEY,

Name,

Salary,

Dnumber,

FOREIGN KEY (Dnumber) REFERENCES **DEPARTMENT** (Dnumber)

)

Relationship Relation Approach

- Create a separate table for the relationship
- Store primary keys of both entities

```
DEPARTMENT(  
    Dnumber PRIMARY KEY,  
    Dname  
)
```

```
EMPLOYEE(  
    Ssn PRIMARY KEY,  
    Name  
)
```

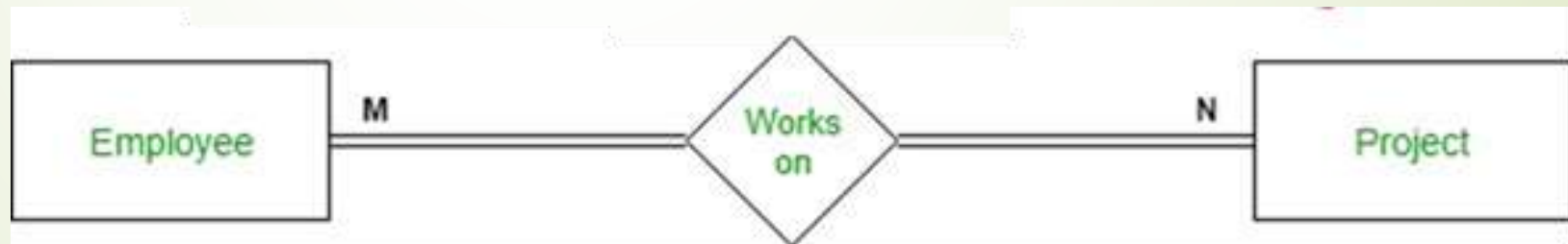
```
WORKS_FOR(  
    Ssn,  
    Dnumber,  
    PRIMARY KEY (Ssn),  
    FOREIGN KEY (Ssn) REFERENCES EMPLOYEE(Ssn),  
    FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber)  
)
```

STEP-5: Mapping of Binary M:N Relationship Types.

- The only option for binary M:N relationships is the cross-reference approach.
- For every binary M:N relationship, we must create a new relation (table).
- In the ER diagram: **Relationship** R = WORKS_ON,

Participating entities: EMPLOYEE and PROJECT

So, we create a new relation (table): WORKS_ON



- 
- Include as foreign key attributes in S the primary keys of the relations that represents the participating entity types.

Eg: Primary Keys of Entities: EMPLOYEE $\rightarrow$ Ssn, PROJECT $\rightarrow$ Project_Number

Add both as **foreign keys in WORKS_ON table:** WORKS_ON(Ssn, Project_Number)

- Their combination will form the primary key of S.

Eg: One employee can work on many projects

One project can have many employees

So, no single attribute is unique.

The combination uniquely identifies each record.

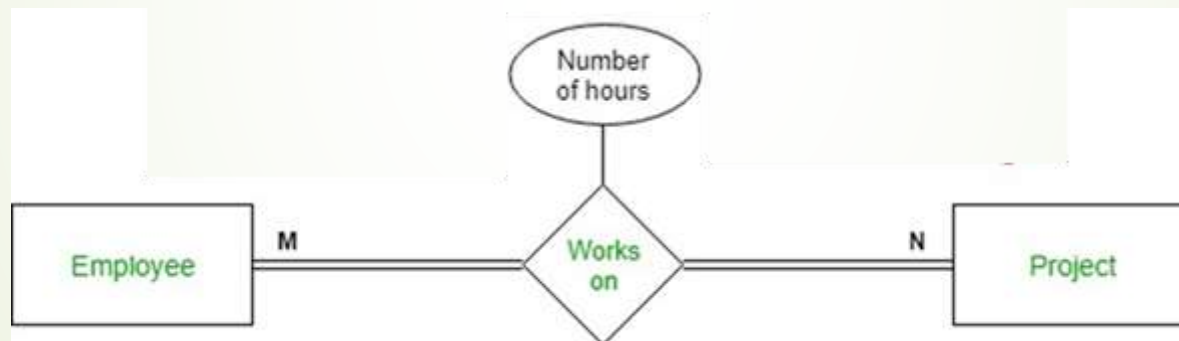
Composite Primary Key: PRIMARY KEY (Ssn, Project_Number)

- Also include any simple attributes of M:N relationship type as attributes of S.

Eg: In the ER diagram:

WORKS_ON has a relationship attribute: Hours (number of hours an employee works on a project)

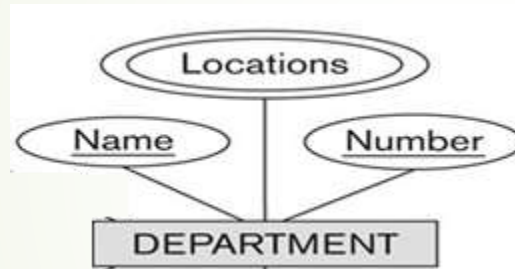
Add it to the new table: WORKS_ON(Ssn, Project_Number, Hours)



Step 6: Mapping of Multivalued Attribute

- For each Multivalued attribute A, create a new relation R.

Eg:



Multivalued attribute: A = Locations

It belongs to: DEPARTMENT entity

Therefore, we **create a new relation:** DEPT_LOCATIONS

- 
- This relation R will include an attribute corresponding to A, plus the primary key attribute K- as a foreign key in R of the relation that represents the entity type or relationship type that has A as a multivalued attribute.

Eg: Primary Key of DEPARTMENT: Number (Department_Number)

Add it as a foreign key in the new table: DEPT_LOCATIONS(Dept_Number, Locations)

- The primary key of R is the combination of A and K.

Eg: One department → many locations

Same location name may appear for different departments

Composite Primary Key: PRIMARY KEY (Dept_Number, Locations)



Step 7: Mapping of N-ARY Relationship Types

The uploaded ER diagram does NOT contain any N-ary ($n > 2$) relationship.

N-ARY Relationship means a relationship involving **more than two entity types**.

- For each n-ary relationship type R , where $n > 2$, create a new relationship relation S to represent R .



Step 7: Mapping of N-ARY Relationship Types

- ▶ For each n -ary relationship type R , where $n > 2$, create a new relationship relation S to represent R .
- ▶ Include as foreign key attributes in S the primary keys of the relations that represent the participating entity types.
- ▶ Also include any simple attributes of the n -ary relationship type as attributes of S .
- ▶ The primary key of S is usually a combination of all the foreign keys that reference the relations representing the participating entity types.



Eg: SUPPLIES (SUPPLIER, PROJECT, PART) The SUPPLIES relationship involves 3 entities, so it is a ternary (3-ary) relationship.

Entities and Their Primary Keys:

SUPPLIER → Supplier_ID

PROJECT → Project_ID

PART → Part_ID

➤ **Create a New Table (Relation) for the N-ary Relationship**

Since an N-ary relationship involves more than two entities, **it cannot be mapped directly into any one entity table.**

Create a new relation to represent the relationship: SUPPLIES

➤ Include Primary Keys of Participating Entities as Foreign Keys

The new table must include the primary keys of all participating entities as foreign keys.

Supplier_ID → FK references SUPPLIER(Supplier_ID)

Project_ID → FK references PROJECT(Project_ID)

Part_ID → FK references PART(Part_ID)

SUPPLIES(Supplier_ID, Project_ID, Part_ID)

Combine the Foreign Keys to Form the Primary Key, Because:

- ❖ A supplier can supply many parts
- ❖ A part can be supplied to many projects
- ❖ A project can receive parts from many suppliers

Only the combination uniquely identifies a record.

Composite Primary Key: PRIMARY KEY (Supplier_ID, Project_ID, Part_ID) This ensures that same supplier cannot supply the same part to the same project more than once

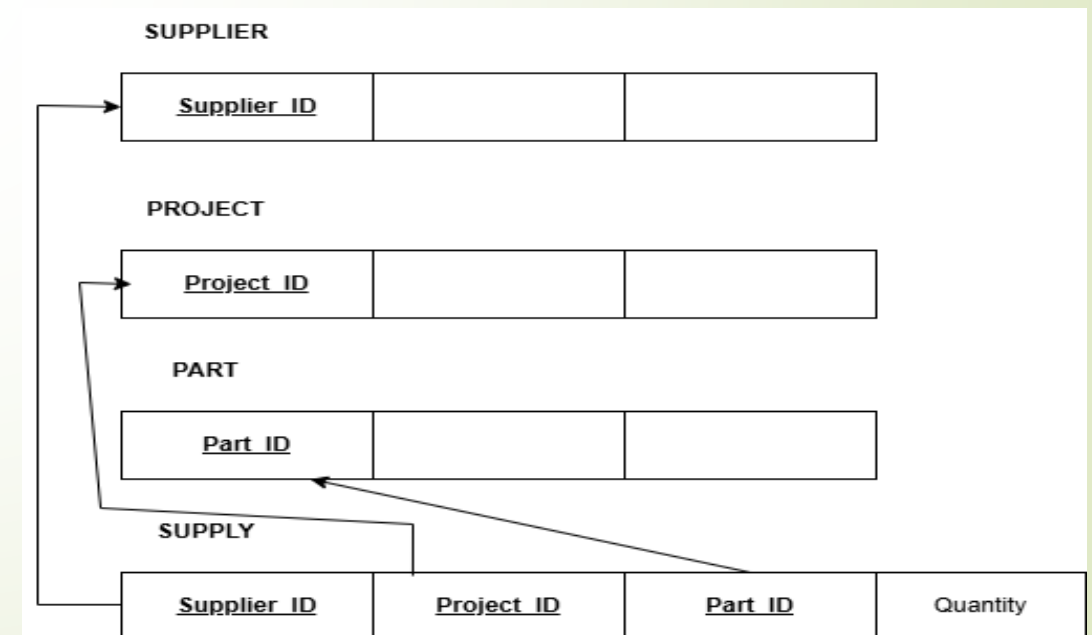
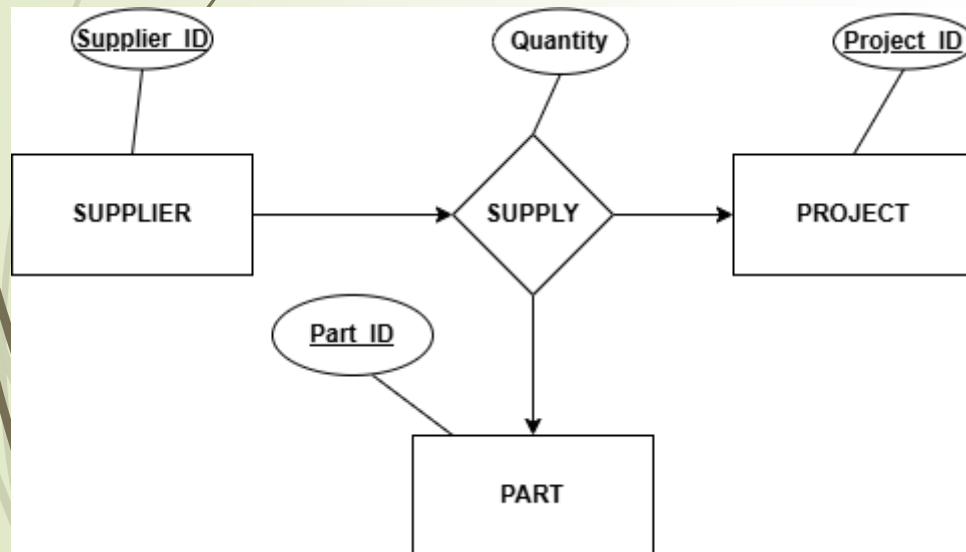
Include Relationship Attributes in the Table

If the relationship has its own attributes, they must be stored in the new table.

Eg: If Quantity is an attribute of the SUPPLIES relationship, we store it in the table.

Add it to the table: SUPPLIES(Supplier_ID, Project_ID, Part_ID, Quantity)

Supplier_ID (FK)	Project_ID (FK)	Part_ID (FK)	Quantity
S1	P1	PR1	100
S2	P2	PR2	50



Correspondence between ER and Relational Models

ER MODEL

Entity type

1:1 or 1:N relationship type

M:N relationship type

n -ary relationship type

Simple attribute

Composite attribute

Multivalued attribute

Value set

Key attribute

RELATIONAL MODEL

Entity relation

Foreign key (or *relationship* relation)

Relationship relation and *two* foreign keys

Relationship relation and n foreign keys

Attribute

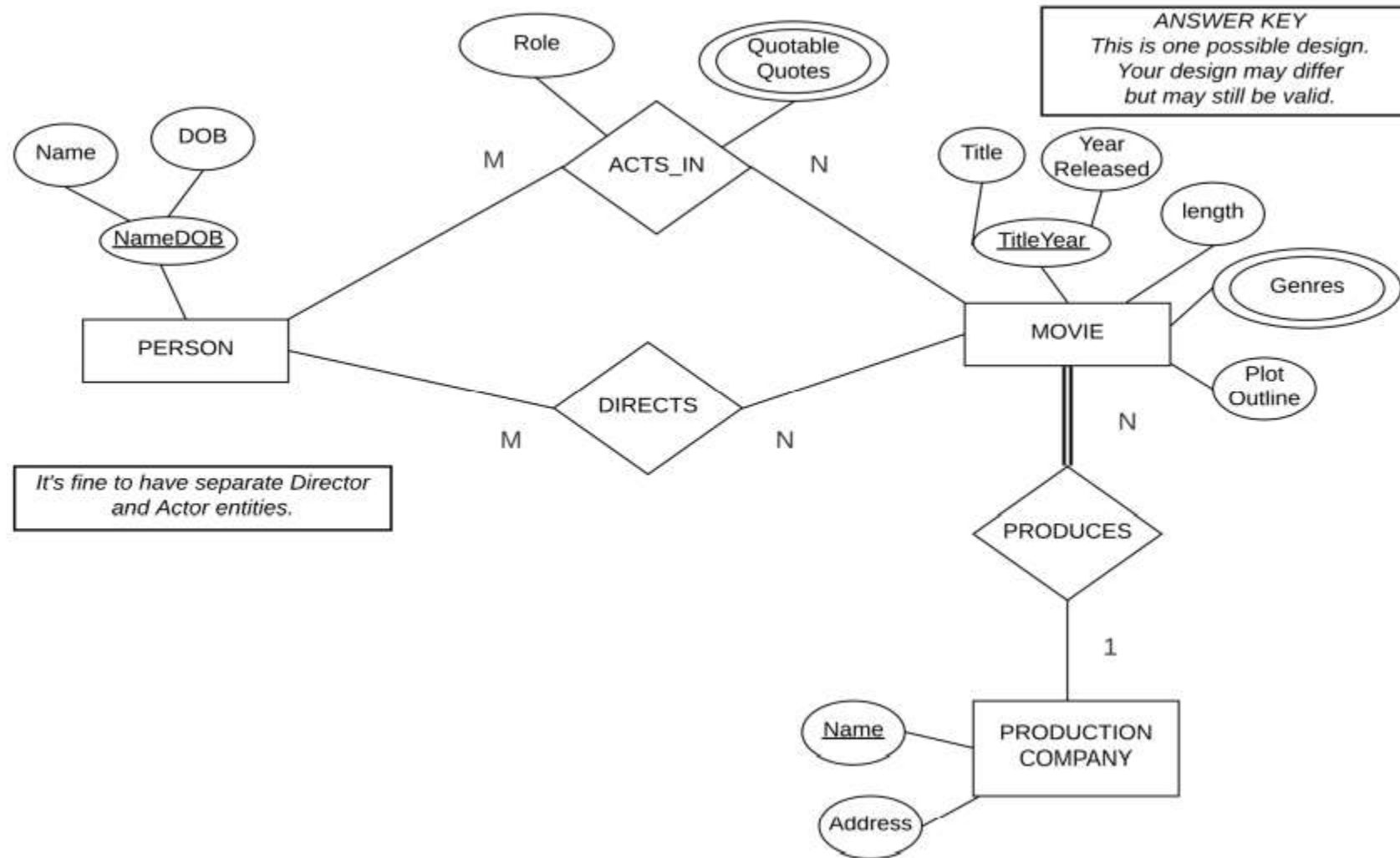
Set of simple component attributes

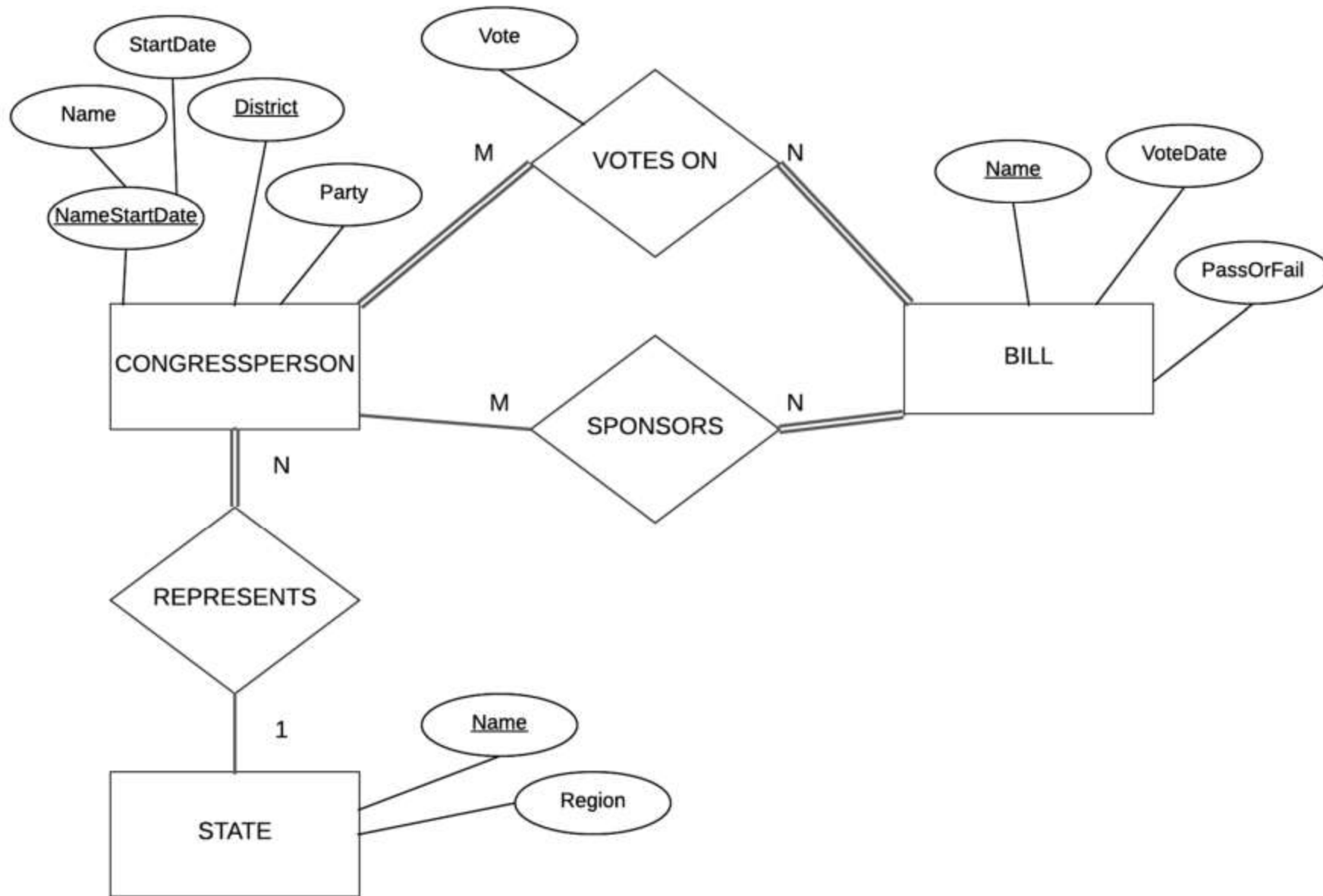
Relation and foreign key

Domain

Primary (or secondary) key

Sample ER diagrams:





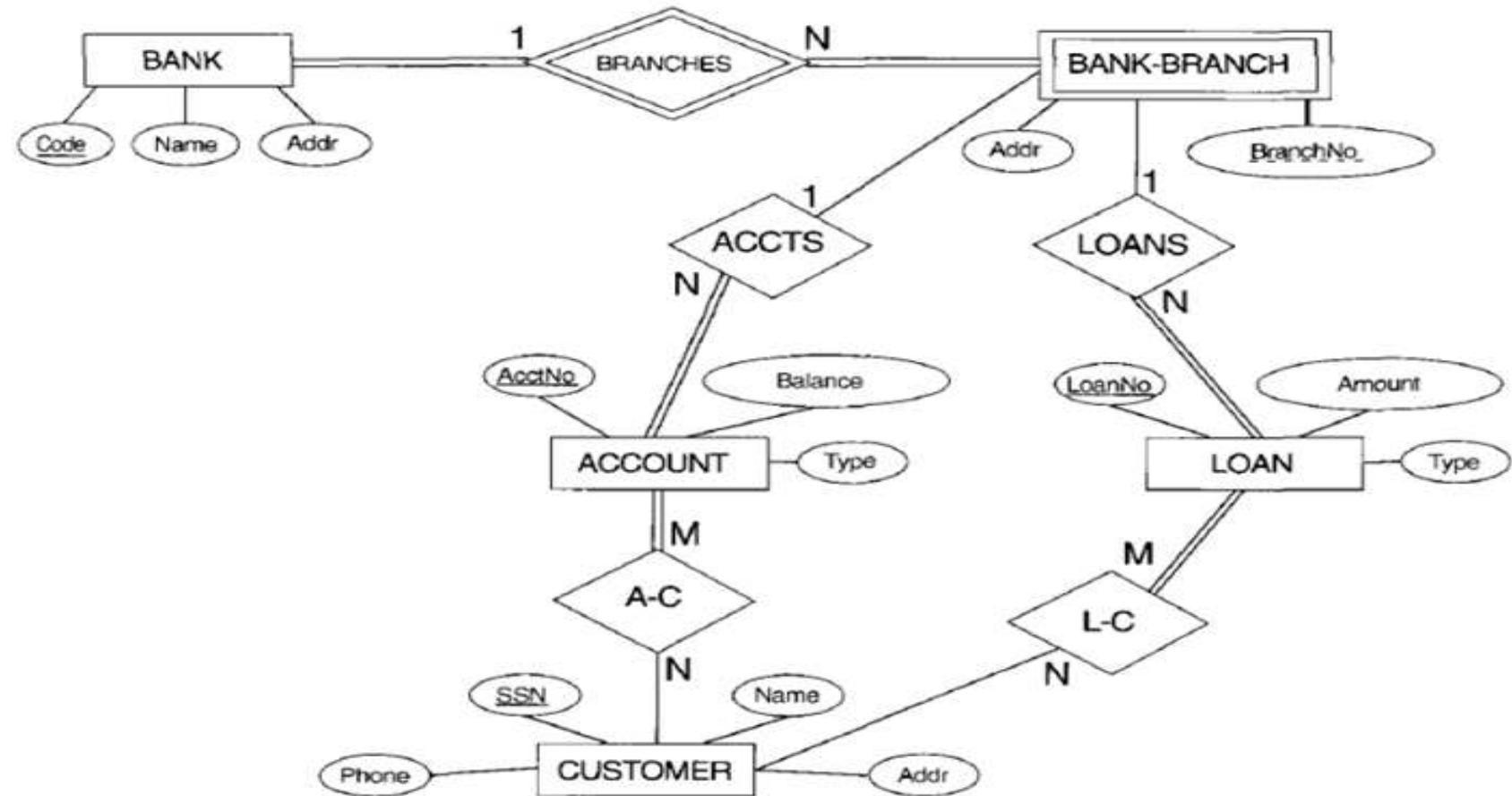


FIGURE 3.18 An ER diagram for a BANK database schema.

An ER schema for a SHIP_TRACKING database.

